

A Quick Tour of Python

Dr. Sumit Kumar Tetarave

ML Lab #1: Part 3

Lists

- Ordered sequence of values
- Written as a sequence of comma-separated values between square brackets
- Values can be of different types
 - usually the items all have the same type

```
>>> lst = [1, 2, 3, 4, 5]
```

```
>>> lst
```

```
[1, 2, 3, 4, 5]
```

```
>>> type(lst)
```

```
<type 'list'>
```

Lists

- List is also a sequence type
 - Sequence operations are applicable

```
>>> fib = [1, 1, 2, 3, 5, 8, 13, 21, 34, 55]
>>> len(fib)
10
>>> fib[3] # Indexing
3
>>> fib[3:] # Slicing
[3, 5, 8, 13, 21, 34, 55]
```

Lists

- List is also a sequence type
 - Sequence operations are applicable

```
>>> [0] + fib # Concatenation
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55]
>>> 3 * [1, 1, 2] # Repetition
[1, 1, 2, 1, 1, 2, 1, 1, 2]
>>> x, y, z = [1, 1, 2] #Unpacking
>>> print (x, y, z )
1 1 2
```

Mutable and Immutable Types

- Tuples and List types look very similar
- However, there is one major difference:
 - Lists are **mutable**: Contents of a list **can be modified**
 - Tuples and Strings are **immutable**: Contents can **not be modified**
- Immutable: An example
s = "Hello"
s[0] = "Y"

More Operations on Lists

- `L.append(x)`
- `L.extend(seq)`
- `L.insert(i, x)`
- `L.remove(x)`
- `L.pop(i)`
- `L.pop()`
- `L.index(x)`
- `L.count(x)`
- `L.sort()`
- `L.reverse()`

`x` is any value, `seq` is a sequence value (list, string, tuple, ...),
`i` is an integer value

An example that uses most of the **list** methods:

```
>>> a = [66.25, 333, 333, 1, 1234.5]
>>> print(a.count(333),
a.count(66.25), a.count('x'))
2 1 0
>>> a.insert(2, -1)
>>> a.append(333)
>>> a
[66.25, 333, -1, 333, 1, 1234.5, 333]
>>> a.index(333)
1
```

```
>>> a.remove(333)
>>> a
[66.25, -1, 333, 1, 1234.5, 333]
>>> a.reverse()
>>> a
[333, 1234.5, 1, 333, -1, 66.25]
>>> a.sort()
>>> a
[-1, 1, 66.25, 333, 333, 1234.5]
```

Using Lists as Stacks and Queues

```
>>> stack = [3, 4, 5]
>>> stack.append(6)
>>> stack.append(7)
>>> stack
[3, 4, 5, 6, 7]
>>> stack.pop()
7
>>> stack
[3, 4, 5, 6]
>>> stack.pop()
6
>>> stack.pop()
5
>>> stack
[3, 4]
```

- However, lists are not efficient for Queues.
 - While appends and pops from the end of list are fast,
 - doing inserts or pops from the beginning of a list is slow (because all of the other elements have to be shifted by one).
- To implement a queue, use **collections.deque** which was designed to have fast appends and pops from both ends. For example:

```
>>> from collections import deque
>>> queue = deque(["Eric", "John", "Michael"])
>>> queue.append("Terry")      # Terry arrives
>>> queue.append("Graham")    # Graham arrives
>>> queue.popleft()           # The first to arrive now leaves
'Eric'
>>> queue.popleft()           # The second to arrive now leaves
'John'
>>> queue                     # Remaining queue in order of arrival
deque(['Michael', 'Terry', 'Graham'])
```


List Comprehensions

- List comprehensions provide a concise way to create lists.

For example, assume we want to create a list of squares, like:

```
>>> squares = []
>>> for x in range(10):
    squares.append(x**2)
```

```
>>> squares
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

We can obtain the same result with:

```
squares = [x**2 for x in range(10)]
```

This is also equivalent to more concise and readable.

```
squares = list(map(lambda x: x**2, range(10)))
```

A list comprehension consists of brackets containing an expression followed by a **for** clause, then zero or more **for** or **if** clauses.

- For example, this list combines the elements of two lists if they are not equal:

```
>>> [(x, y) for x in [1,2,3] for y in [3,1,4] if x != y]
[(1, 3), (1, 4), (2, 3), (2, 1), (2, 4), (3, 1), (3, 4)]
```

```
>>> combs = []
>>> for x in [1,2,3]:
    for y in [3,1,4]:
        if x != y:
            combs.append((x, y))
>>> combs
```

Note how the order of the **for** and **if** statements is the same in both

```
>>> [(x, y) for x in [1,2,3] for y in [3,1,4] if x != y]
[(1, 3), (1, 4), (2, 3), (2, 1), (2, 4), (3, 1), (3, 4)]
```

```
>>> combs = []
>>> for x in [1,2,3]:
    for y in [3,1,4]:
        if x != y:
            combs.append((x, y))
>>> combs
```

Note how the order of the `for` and `if` statements is the same in both

If the expression is a tuple (e.g. the (x, y) in the previous example), it must be parenthesized.

```
>>> vec = [-4, -2, 0, 2, 4]
# create a new list with the values doubled
>>> [x*2 for x in vec]
[-8, -4, 0, 4, 8]
# filter the list to exclude negative numbers
>>> [x for x in vec if x >= 0]
[0, 2, 4]
# apply a function (e.g. abs()) to all the elements
>>> [abs(x) for x in vec]
[4, 2, 0, 2, 4]
# call a method (e.g. strip()) on each element
>>> freshfruit = [' banana', ' loganberry ',
'passion fruit ']
>>> [weapon.strip() for weapon in freshfruit]
['banana', 'loganberry', 'passion fruit']
```

```
freshfruit = [' xbanana_', ' xyz_loganberry.', '10passion fruit5 ']
```

```
[weapon.strip('_ . 5 10 x xyz_') for weapon in freshfruit]
```

```
['banana', 'loganberr', 'passion fruit']
```

```
# Till 5- create a list of 2-tuples like (number, square)
>>> [(x, x**2) for x in range(6)]
[(0, 0), (1, 1), (2, 4), (3, 9), (4, 16), (5, 25)]
```

the tuple must be parenthesized, otherwise an error is raised

```
>>> [x, x**2 for x in range(6)]
File "<stdin>", line 1, in ?
[x, x**2 for x in range(6)]
      ^
```

SyntaxError: invalid syntax

flatten a list using a listcomp with two 'for'

```
>>> vec = [[1,2,3], [4,5,6], [7,8,9]]
```

Output: [1, 2, 3, 4, 5, 6, 7, 8, 9]

```
>>> [num for elem in vec for num in elem]
```

```
flattenList = []
for elem in vec:
    for num in elem:
        flattenList.append(num)
flattenList
```

Nested List Comprehensions

- The initial expression in a list comprehension can be any arbitrary expression, including another list comprehension.
- Consider the following example of a 3x4 matrix implemented as a list of 3 lists of length 4:

```
>>> matrix = [ [1, 2, 3, 4],  
                [5, 6, 7, 8],  
                [9, 10, 11, 12],]
```

The list comprehension will transpose rows and columns:

```
transposed = []  
for i in range(4):  
    col=[]  
    for row in matrix:  
        col.append(row[i])  
    print(col)  
    transposed.append(col)  
transposed
```

```
[1, 5, 9]  
[2, 6, 10]  
[3, 7, 11]  
[4, 8, 12]  
[[1, 5, 9], [2, 6, 10], [3, 7, 11], [4, 8, 12]]
```

```
>>> transposed = []  
>>> for i in range(4):  
        transposed.append([row[i] for row in matrix])  
>>> transposed  
[[1, 5, 9], [2, 6, 10], [3, 7, 11], [4, 8, 12]]
```

```
>>> [[row[i] for row in matrix] for i in range(4)]  
[[1, 5, 9], [2, 6, 10], [3, 7, 11], [4, 8, 12]]
```

Practice
on **map()**,
lambda(),
zip(), etc.

In the real world, you should prefer built-in functions to complex flow statements. The `zip()` function would do a great job for this use case.

```
list(zip(*matrix))
```

```
[(1, 5, 9), (2, 6, 10), (3, 7, 11), (4, 8, 12)]
```

```
list(zip(matrix))
```

```
[([1, 2, 3, 4],), ([5, 6, 7, 8],), ([9, 10, 11, 12],)]
```

- **zip() function in Python** combines multiple iterables such as **lists, tuples, strings**, etc, into a single iterator of tuples.
- Each tuple contains elements from the input iterables that are at the same position.
- For example: to pair **student names** with their **test scores**:

```
names = ['John', 'Alice', 'Bob', 'Lucy']  
scores = [85, 90, 78, 92]
```

```
res = zip(names, scores)  
print(list(res))
```

```
[('John', 85), ('Alice', 90), ('Bob', 78), ('Lucy', 92)]
```

Parameters: `zip(*iterables)`

`*iterables` refers to one or more iterable objects (like lists, tuples, etc.) that we want to combine.

The function pairs elements from these iterables into tuples **based on their positions**.

```
matrix
```

```
[[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]]
```

```
list(zip(matrix))
```

```
[[([1, 2, 3, 4],), ([5, 6, 7, 8],), ([9, 10, 11, 12],)]
```

```
display(*matrix)
```

```
[1, 2, 3, 4]
```

```
[5, 6, 7, 8]
```

```
[9, 10, 11, 12]
```

```
list(zip(*matrix))
```

```
[(1, 5, 9), (2, 6, 10), (3, 7, 11), (4, 8, 12)]
```

- ✓ The **asterisk *** operator unpacks the list matrix, and
- ✓ **zip()** then recombines the elements as columns

Python `map()` and `Lambda` function

- The **`map()` function** is used to apply a given function to every item of an **iterable**, such as a `list` or `tuple`, and returns a **map object** (which is an iterator).
- A simple example of using **`map()`** to convert a list of strings into a list of integers.

```
s = ['1', '2', '3', '4']  
res = map(int, s)  
print(list(res))
```

```
[1, 2, 3, 4]
```

Explanation: Here, we used the built-in **`int`** function to convert each string in the list `s` into an integer. The **`map()`** function takes care of applying **`int()`** to every element

Parameter: *map(function, iterable)*

- **function:** The function we want to apply to every element of the iterable.
- **iterable:** The iterable whose elements we want to process.

Lambda can have any number of arguments but only one expression.

- The expression is evaluated and returned.
- Lambda functions are anonymous, meaning they are not bound to a name.

1. Simple Lambda Function:

```
add = lambda x, y: x + y  
print(add(2, 3)) # Output: 5
```

2. Using Lambda with `map()`:

```
numbers = [1, 2, 3, 4]  
squared = list(map(lambda x: x ** 2, numbers))  
print(squared) # Output: [1, 4, 9, 16]
```

```
# Using Lambda
sq = lambda x: x ** 2
print(sq(3))
```

```
# Using def
def sqdef(x):
    return x ** 2
print(sqdef(3))
```

Feature	lambda Function	Regular Function (def)
Definition	Single expression with lambda.	Multiple lines of code.
Name	Anonymous (or named if assigned).	Must have a name.
Statements	Single expression only.	Can include multiple statements.
Documentation	Cannot have a docstring.	Can include docstrings.
Reusability	Best for short, temporary functions.	Better for reusable and complex logic.

Vacuum Cleaner (Simple Reflex AI Agent)

1. `updateState(state, action)`
2. `getAction(state)`
3. `bool_all_rooms_clean(state)`
4. `run_vacuum(start_state)`

- The simulation has four main functions:

```
•[9]: start_state = ['R', [1, 1]]  
print(f"\nPart1. Start in room R\nStart State: {start_state}")  
run_vacuum(start_state)
```

```
Part1. Start in room R  
Start State: ['R', [1, 1]]  
cycle:0 action=SUCK  
cycle:1 action=MOVELEFT  
cycle:2 action=SUCK  
All rooms clean!
```

```
state=['R', [1, 0]]  
state=['L', [1, 0]]  
state=['L', [0, 0]]
```

Function 4. run_vacuum(start_state)

- Runs the simulation:
 - Repeatedly determines the action using getAction.
 - Updates the state using updateState.
 - Stops when all rooms are clean or if the program enters too many cycles, indicating a potential bug.

```
[4]: def run_vacuum(start_state):
    state = start_state
    loop_count = 0 # counter to stop program when it goes wrong

    # run the simulation until all rooms clean
    while not bool_all_rooms_clean(state):
        action = getAction(state)
        state = updateState(state, action)

        # show state and stop if too many loops
        print(f"cycle:{loop_count}\taction={action}\t\tstate={state}")
        loop_count += 1

        if loop_count > 20:
            print(
                "Uh oh ! Code stuck in a state ... program terminated")
            return

    print("All rooms clean!")
    return
```

```
•[3]: def bool_all_rooms_clean(state):  
  
    # If all room are clean then  
    # we Return True or else False.  
    if state[1][0]==0 and state[1][1]==0:  
        return True  
    else:  
        return False
```

Function 3: **bool_all_rooms_clean(state)**

- Checks if all rooms are clean:
 - Returns **True** if both rooms are clean ([0, 0]).
 - Returns **False** otherwise.

```
•[1]: def getAction(state):  
    Curr_State=state  
  
    # We make use of Conditional Statement To Get  
    # the Next Action Given current State.  
    if Curr_State[0]=='R' and Curr_State[1][1]==1:  
        return "SUCK"  
    elif Curr_State[0]=='L' and Curr_State[1][0]==1:  
        return "SUCK"  
    elif Curr_State[0]=='R' and Curr_State[1][1]==0:  
        return "MOVELEFT"  
    elif Curr_State[0]=='L' and Curr_State[1][0]==0:  
        return "MOVERIGHT"  
    else:  
        return None
```

Function 2: **getAction(state)**

Determines the next action the vacuum cleaner should take based on the current state:

- **SUCK**: Cleans the room if dirty.
- **MOVELEFT/MOVERIGHT**: Moves to the adjacent room if the current room is clean.
- **None**: Stops if no valid action is found.

```
•[2]: def updateState(state, action):  
    Curr_State=state  
  
    # Here, given Current State and Action  
    # we Update the State for Next Action.  
    if Curr_State[0]=='R' and action=="SUCK":  
        Curr_State[1][1]=0  
    elif Curr_State[0]=='L' and action=="SUCK":  
        Curr_State[1][0]=0  
    elif Curr_State[0]=='R' and action=="MOVELEFT":  
        Curr_State[0]='L'  
    elif Curr_State[0]=='L' and action=="MOVERIGHT":  
        Curr_State[0]='R'  
  
    return Curr_State
```

Function 1: **updateState(state, action)**

- Updates the vacuum cleaner's state based on its action:
 - If cleaning (SUCK), the room is marked clean (0).
 - If moving (MOVELEFT/MOVERIGHT), the vacuum cleaner's position is updated.